State and Events

Jaxer provides four state contexts, with each state having its own container to persist values. To illustrate session state: when a user closes and then reopens a browser window of a web application yet remains logged in, the state of that user's session remains in effect.

API Links

Jaxer
Jaxer.Container
Jaxer.SessionManager

Contexts

The four states, from the narrowest scope to the broadest, are:

- `Jaxer.sessionPage`: Values stored in the private context of a single page and belong to a specific session, and as such will endure over the entirety of that session, expiring when the user's session expires.
- `Jaxer.session`: A session is automatically created within the context of an application, expiring when the user's session expires. Session containers are not shared among applications so if a user is working with two distinct Jaxer web applications, that user has a separate session container for each application.
- `Jaxer.page`: Persist values within the private context of a single page and shared within the application and among sessions. This is analogous to a static member of a class in Java—the container belongs to the page yet the values are static within the application. This container only expires when the Jaxer server is shutdown.
- `Jaxer.application`: Stores values accessible within a single application. The application container is always availabl when any user accesses a page belonging to an application. An application container might be used to cache shared rarely changed data originally loaded from disk or a database; your application code is reponsible for managing the cache and updating the session as necessary. This container and its values expire only when the Jaxer server is shutdown.

Using the State Containers

Even though each state is stored internally as a Jaxer.Container object, you don't interact though the Container API; instead, your co treats each state as if it were a normal JavaScript associative array:

```
1. Jaxer.session["username"] = "Bullwinkle";
2. var username = Jaxer.session["username"];
3. var exists = ("username" in Jaxer.session);
4. delete Jaxer.session["username"];
5. Jaxer.session = {};
6. Jaxer.application["app_key"] = "s0m3h4sh";
7. var app_key = Jaxer.application["app_key"];
8. var akExists = ("app_key" in Jaxer.application);
```

Passing Data to the Client

The Jaxer.clientData container is used to send data from the server to the client at the end of server-side page processing. Using `Jaxer.clientData` is a simpler technique to pass data to the browser at the initial page request than using hidden form elements to pass values.

When Jaxer begins to process a page at the server, `Jaxer.clientData` object is empty: `{}` . If you set any property on this object, th entire object will be serialized to JSON at the end of processing on the server, and will be automatically deserialized by Jaxer in the browser. This data is referenced in the browser also as `Jaxer.clientData`. If no data was added at the server, `Jaxer.clientData` is not created in the browser.

Session State and Scaling

Jaxer stores session state in the database it's configured to use. If scalability is required, it's best to configure Jaxer to use an externa database instead of the internal SQLite database. Multiple Jaxer servers will then be able to share session data upon requests.

```
01. <html>
02.     <head>
03.         <title>Working with Session State</title>
04.         <script type="text/javascript" runat="server-proxy">
05.
06.             function loginUser(username, password) {
07.
08.                 if ((username == "admin") && (password == "sesame")) {
09.                     if (Jaxer.session["username"] == "admin") {
10.                         Jaxer.Log.info("Admin user is already logged in.");
11.                     } else {
12.                         Jaxer.Log.info("Logging in admin user.");
13.                         Jaxer.session["username"] = username;
14.                     }
15.                 }
16.             }
17.             loginUser.proxy = true;
18.
19.             function performAdminTask() {
20.
21.                 if (Jaxer.session["username"] == "admin") {
22.                     return "The admin task has been completed.";
23.                 }
24.             }
25.             performAdminTask.proxy = true;
26.
27.             function logoutUser() {
28.
```

```
            if ("username" in Jaxer.session) {
                delete Jaxer.session["username"];
            }
        }
        logoutUser.proxy = true;

    </script>
    <script type="text/javascript" runat="client">

        window.onload = function() {

            loginUser("admin", "sesame");
            var result = performAdminTask();
            window.alert("result: " + result);
            logoutUser();
        };

    </script>
  </head>
</html>
```

Jaxer and Event Handlers

Typically the interesting events in a web application occur on the client: a document finishes loading, the user clicks a link or submits a form, or hovers her mouse over an image. Because the events happen after the server completes its processing by default the events aren't available to handle with Jaxer.

Your code can, though, in unobtrusive fashion, assign handlers for events dynamically in processing the DOM before it gets streamed to the user's browser. Since one powerful use case for Jaxer is to build pages dynamically on the server `Jaxer.setEvent()` is a convenient way to assign such handlers.

```
function clickMe() {
  // do something interesting here
 }

var inp = Document.getElementById('someElement');

// your first instinct is the following line but it doesn't work:
// inp.onclick = clickMe;

// instead, use setEvent:
Jaxer.setEvent("onclick", input, clickMe);

// the generated client side code you'll see using View Source
// should be similar to:
<div id="someElement" onclick="clickMe();"></div>
```

Event Listeners

Your Jaxer apps can leverage `window.addEventListener` to hook into events in the browser in an unobtrusive manner. Jaxer adds the following events to those you can listen on:

1. serverload
2. aftercallbackprocessing
3. load

```
<html>
   <head>
      <script runat="server">
         /* Simple listeners for each of the three events */
         function OnServerLoadListener() { Jaxer.Log.info("OnServerLoadListene
         ↵ called") }
         function OnLoadListener() { Jaxer.Log.info("OnLoadListener called") }
         function OnAfterCallbackProcessingListener() {
         ↵ Jaxer.Log.info("OnAfterCallbackProcessingListener called") }

         /* Add the listeners */
         window.addEventListener("serverload", OnServerLoadListener, false);
         window.addEventListener("load", OnLoadListener, false);
         window.addEventListener("aftercallbackprocessing",
         ↵ OnAfterCallbackProcessingListener, false);

         window.onload = function() { Jaxer.Log.info('window.onload called') }
         window.onserverload = function() { Jaxer.Log.info('window.onserverloa
         ↵ called') };
         window.onaftercallbackprocessing = function() {
         ↵ Jaxer.Log.info("window.onaftercallbackprocessing") };
      </script>
   </head>
   <body onserverload="Jaxer.Log.info('BODY.onserverload called')"
   ↵ onaftercallbackprocessing="Jaxer.Log.info('BODY.onaftercallbackprocessi
      called')" onload="alert('On the client! The body.onload is never called
      server-side.')">

<h2>Results in the Jaxer Log should be:</h2>
<pre>
[INFO] [JS Framework] [framework.] OnServerLoadListener called
[INFO] [JS Framework] [framework.] BODY.onserverload called
[INFO] [JS Framework] [framework.] OnLoadListener called
[INFO] [JS Framework] [framework.] window.onload called
[INFO] [JS Framework] [framework.] OnAfterCallbackProcessingListener called
```

```
[INFO] [JS Framework] [framework.] BODY.onaftercallbackprocessing called
</pre>
</body>
</html>
```

Note that the `window.onserverload` and `window.onaftercallbackprocessing` event handlers have been replaced with values from the body element. Similarly, making the `window.on<event>` assignment after the body element will replace handlers specified in body attributes.

The behavior of events listeners assigned to the `load` event is controlled through the `Jaxer.Config.ONLOAD_ENABLED` configuration optic as well as per-request via the `Jaxer.onloadEnabled` runtime value (which defaults to true). As always, the body's `onload` event handle is never executed server-side.